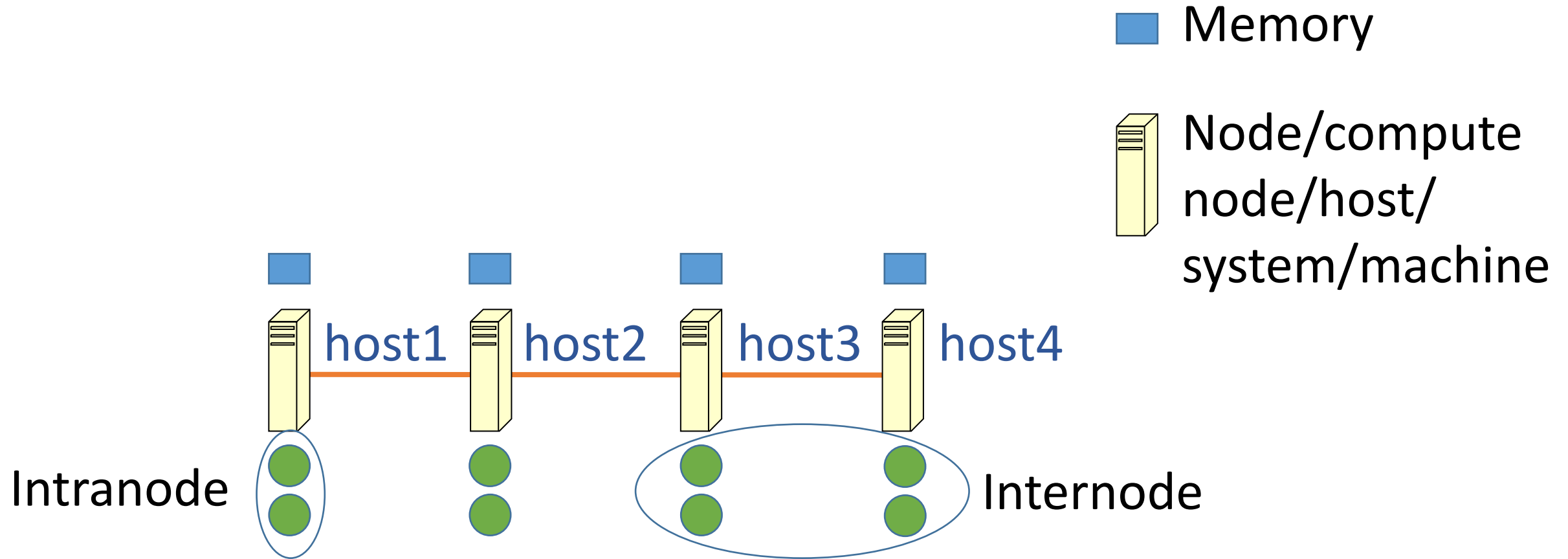


P2P Communication

Lecture 4

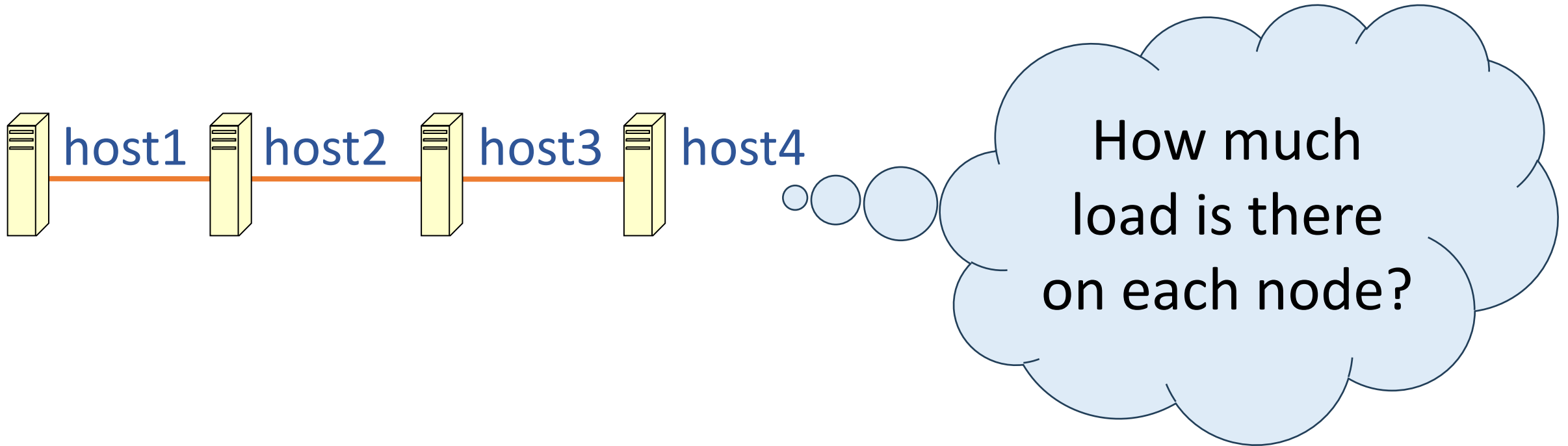
Jan 14, 2026

MPI Program Execution



```
mpirun -n 8 -hosts host1,host2,host3,host4 ./exe
```

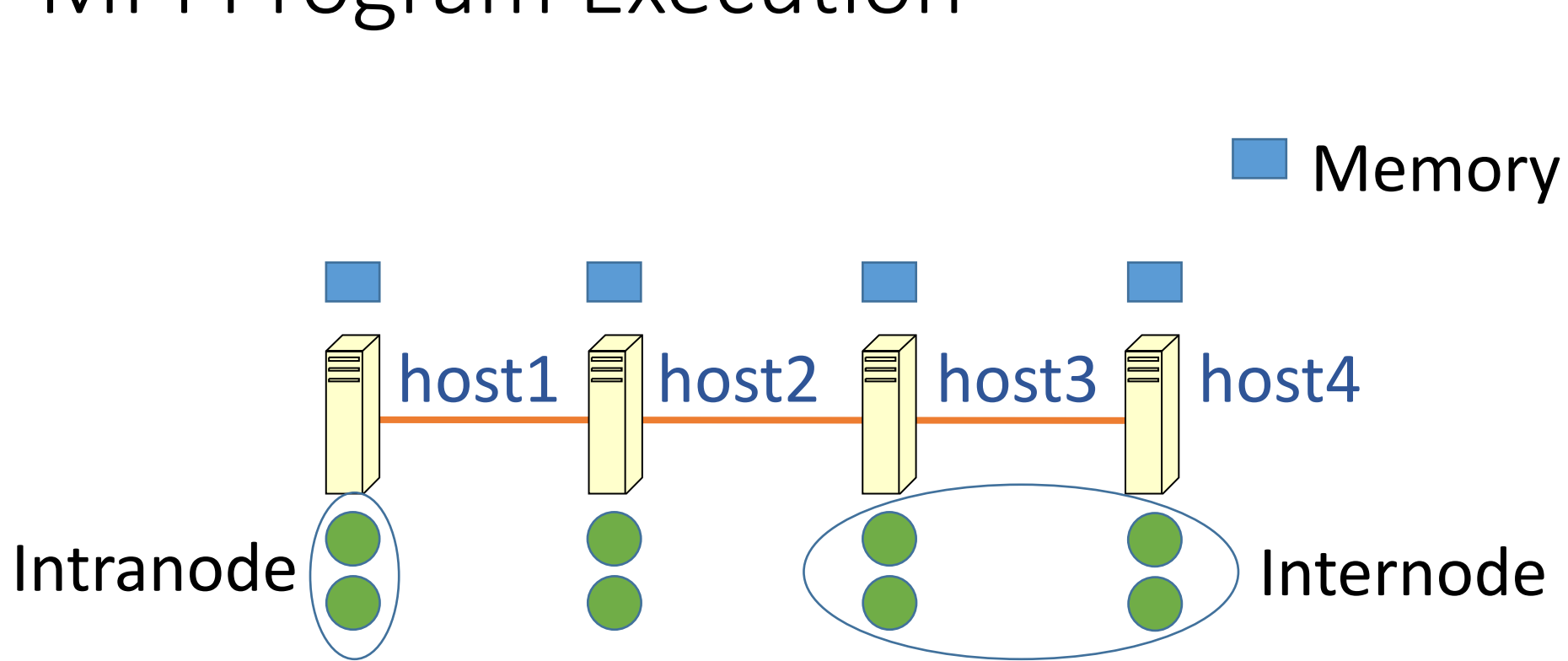
Execution on Beowulf/Unmanaged Cluster



Network and Load-Aware Resource Manager for MPI Programs

<https://dl.acm.org/doi/10.1145/3409390.3409406> [Homework]

MPI Program Execution



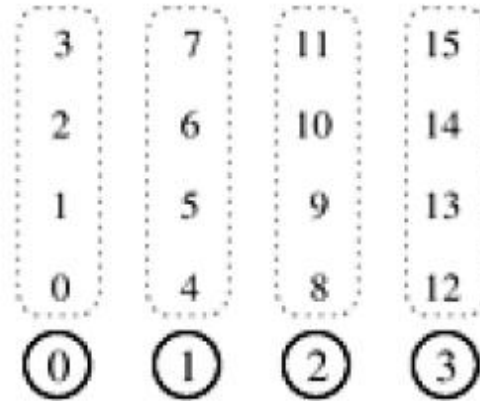
hostfile

```
cn023  
cn024  
cn025  
cn026  
cn023  
cn024  
cn025  
cn026
```

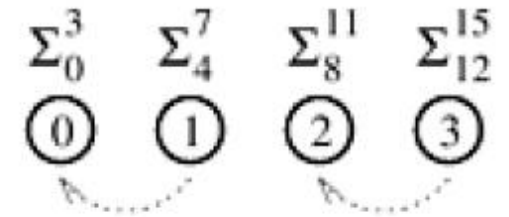
```
mpiexec -n 8 -hosts host1,host2,host3,host4 ./exe
```

Homework

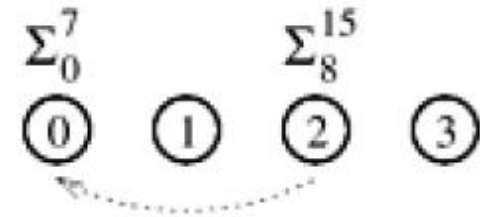
Analyze the communication endpoints for the optimized algorithm of parallel sum for round-robin and sequential placement of 4 processes (ppn=1) and 8 processes (ppn=2) on 2D mesh network.



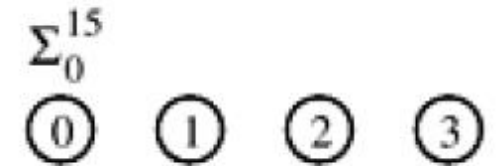
(a)



(b)

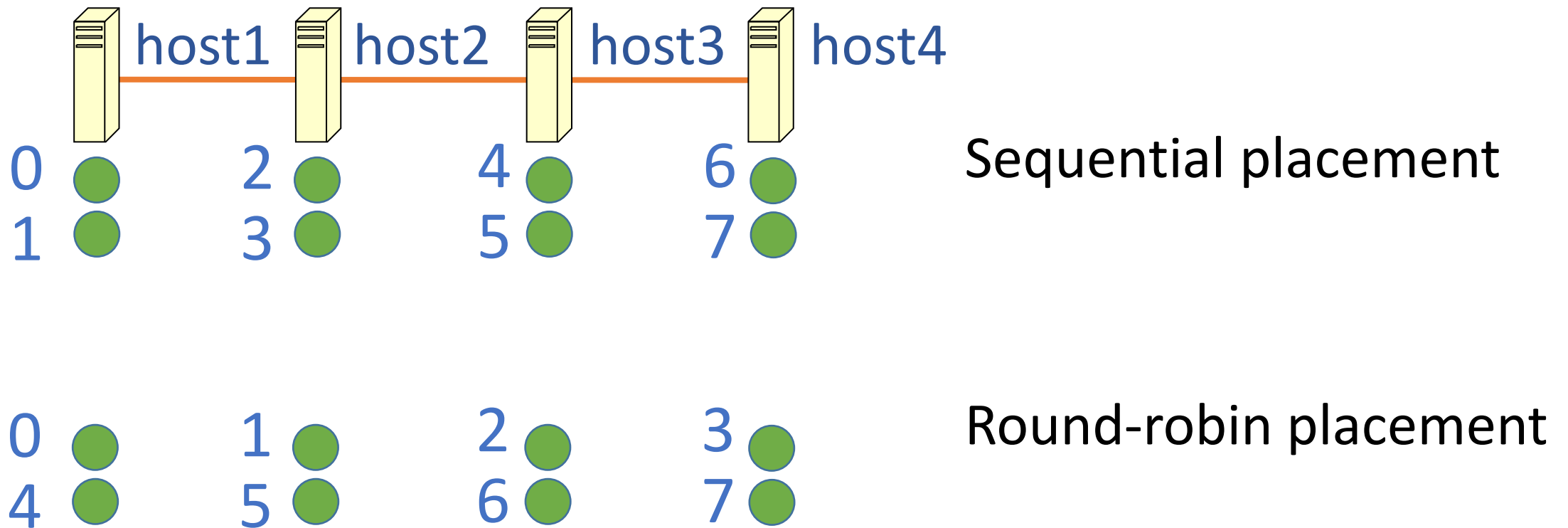


(c)

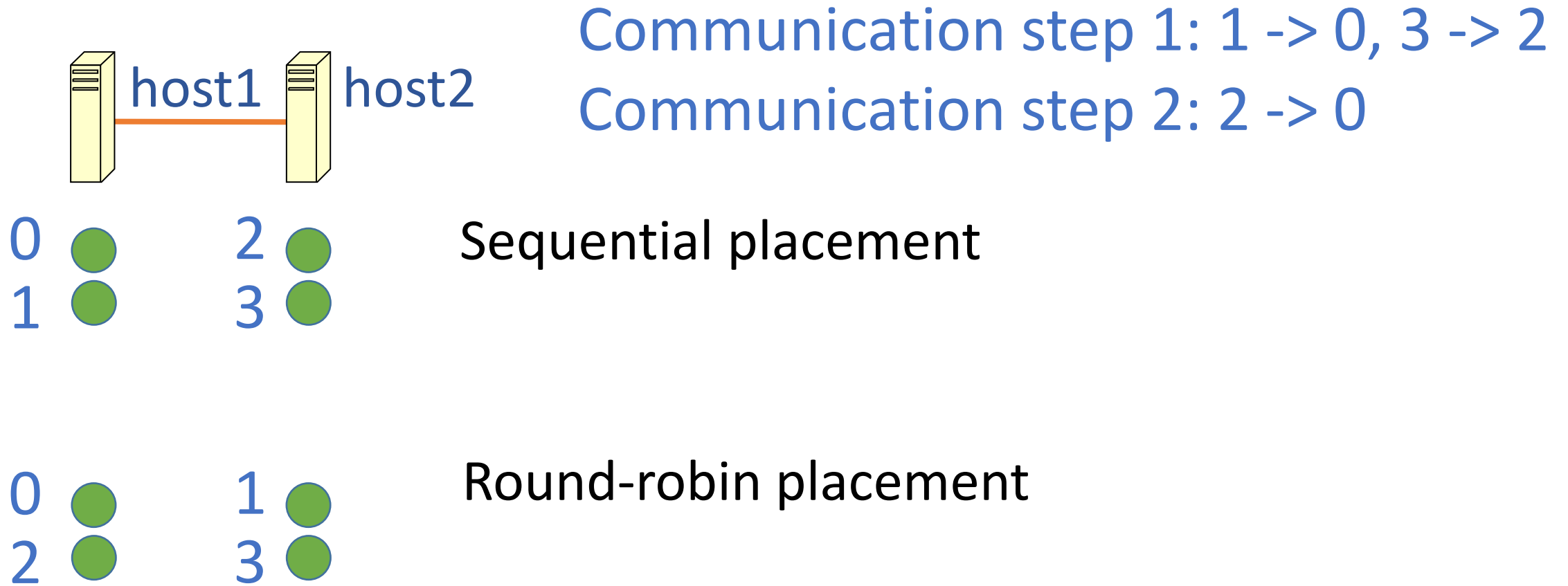


(d)

Process Placement - Parallel Sum (Optimized)



Parallel Sum (Optimized) on 4 Processes



Number of Hops on 4 Processes



Communication step 1: 1 → 0, 3 → 2

Communication step 2: 2 → 0



Sequential placement

Communication step 1 #hops: 0, 0

Max: 0

Communication step 2 #hops: 1

Max: 1

Sum: 1



Round-robin placement

Communication step 1 #hops: 1, 1

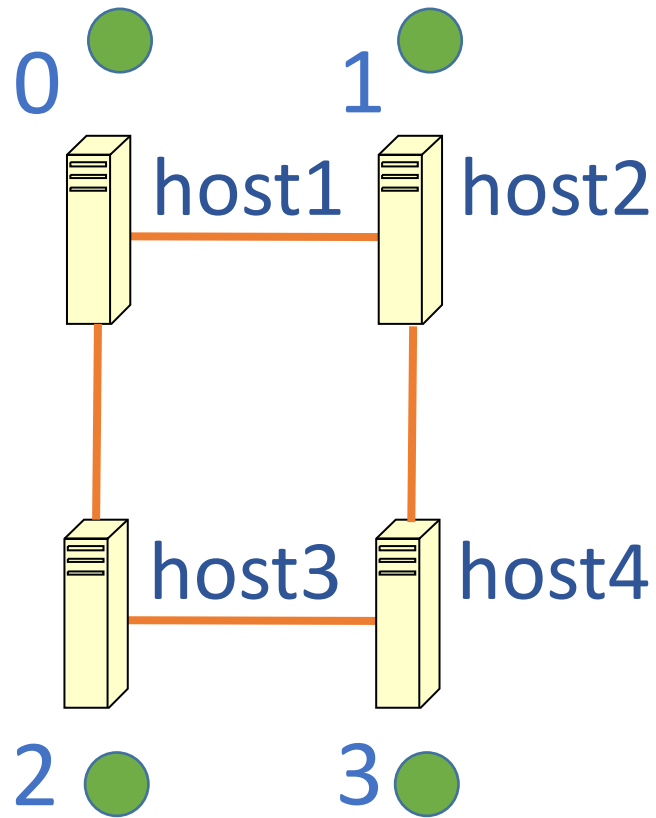
Max: 1

Communication step 2 #hops: 0

Max: 0

Sum: 1

Parallel Sum on 4 processes in a 2D Mesh



Communication step 1: 1 $\rightarrow$ 0, 3 $\rightarrow$ 2

Communication step 2: 2 $\rightarrow$ 0

Sequential placement

Communication step 1 #hops: 1, 1

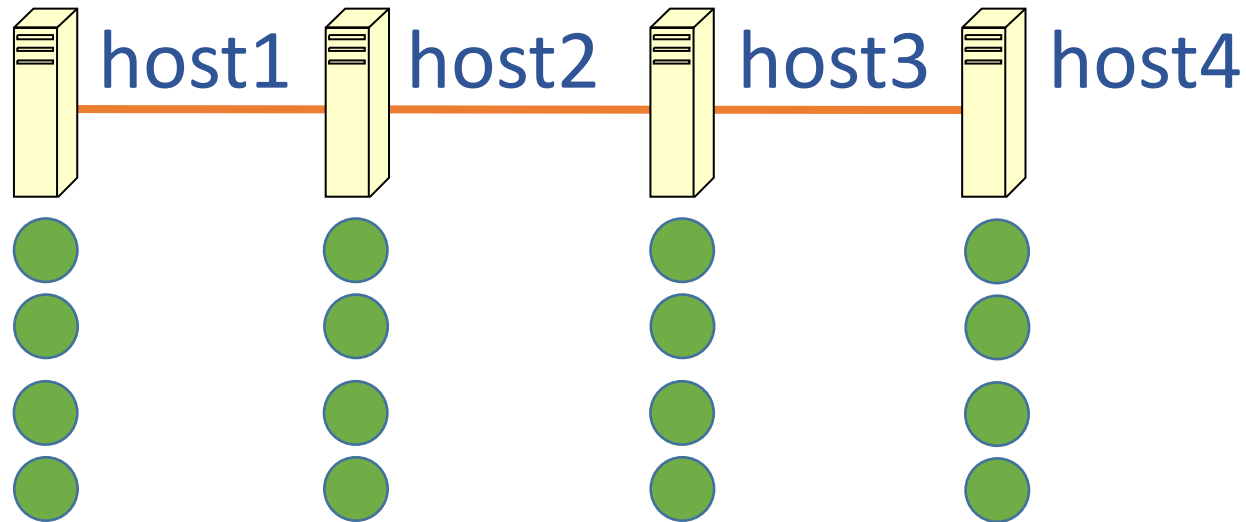
Communication step 2 #hops: 1

Max: 1

Max: 1

Sum: 2

Homework: Analyze #Hops



Analyze maximum #hops
and total hops for $np = 8$
and $np=16$ ($ppn=2, 4$) for

Sequential placement

vs.

Round-robin placement

MPI Reference Material

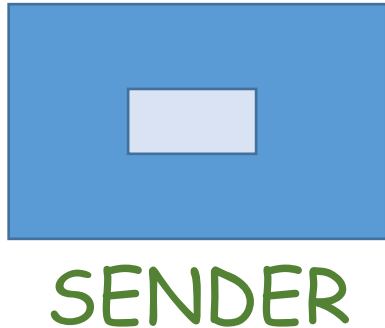
- Marc Snir, Steve W. Otto, Steven Huss-Lederman, David W. Walker and Jack Dongarra, MPI - The Complete Reference, Second Edition, Volume 1, The MPI Core.
- William Gropp, Ewing Lusk, Anthony Skjellum, Using MPI: portable parallel programming with the message-passing interface, 3rd Ed., Cambridge MIT Press, 2014.
- MPI Report

<https://www.mpi-forum.org/docs/mpi-5.0/mpi50-report.pdf>

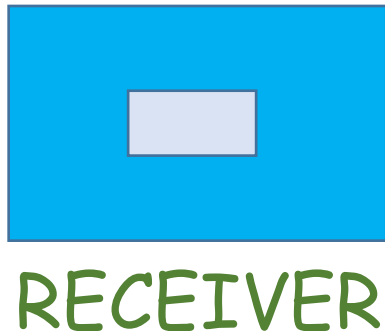
P2P/Direct Communication

Blocking send and receive

MPI_Send



MPI_Recv



```
int MPI_Send (const void *buf, int
count, MPI_Datatype datatype, int
dest, int tag, MPI_Comm comm)
```

Tags should match

```
int MPI_Recv (void *buf, int count,
MPI_Datatype datatype, int source,
int tag, MPI_Comm comm,
MPI_Status *status)
```

Example

```
int MPI_Send (const void *buf, int count, MPI_Datatype  
datatype, int dest, int tag, MPI_Comm comm)
```

Send 1 INT from rank 0 to rank 1

```
// Initialization
```

```
if (myrank == 0)
```

```
    MPI_Send (buf, 1, MPI_INT, 1, 1, MPI_COMM_WORLD);
```

```
else if (myrank == 1)
```

```
    MPI_Recv (buf, 1, MPI_INT, 0, 1, MPI_COMM_WORLD, &status);
```

Example

Certain scenarios are implementation-specific

```
int MPI_Send (const void *buf, int count, MPI_Datatype  
datatype, int dest, int tag, MPI_Comm comm)
```

Will this code execute?

```
// Initialization
```

```
MPI_Send (buf, 1, MPI_INT, 1, 1, MPI_COMM_WORLD);  
MPI_Recv (buf, 1, MPI_INT, 0, 1, MPI_COMM_WORLD, &status);
```

Example

BLOCKING

```
int MPI_Send (const void *buf, int count, MPI_Datatype  
datatype, int dest, int tag, MPI_Comm comm)
```

Will this code execute?

- `mpirun -np 2 ./a.out <count>`
- `mpirun -np 3 ./a.out <count>` What is the specified communication pattern?

```
MPI_Send (buf, count, MPI_INT, (rank+1)%2, 1, MPI_COMM_WORLD);  
MPI_Recv (buf, count, MPI_INT, (rank+1)%2, 1, MPI_COMM_WORLD,  
&status);
```

Example

BLOCKING

```
int MPI_Send (const void *buf, int count, MPI_Datatype  
datatype, int dest, int tag, MPI_Comm comm)
```

Will this code execute?

- `mpirun -np 2 ./a.out <count>` 0 and 1 are blocked, may execute! (more later)
- `mpirun -np 3 ./a.out <count>` Matching receive for send from 2 is missing.

```
MPI_Send (buf, count, MPI_INT, (rank+1)%2, 1, MPI_COMM_WORLD);
```

```
MPI_Recv (buf, count, MPI_INT, (rank+1)%2, 1, MPI_COMM_WORLD, &status);
```

This is an incorrect/unsafe code

Executing MPI programs

- Check ``which mpicc`` and ``which mpiexec``
 - Update PATH environment variable
- Compile
 - `mpicc -o filename filename.c`
- Run
 - `mpiexec -np 4 -f hostfile filename` [Often “No such file” error]
 - `mpiexec -np 4 -f hostfile ./filename`

Simple Send/Recv Code (sendmessage.c)

```
//From https://www.mpi-forum.org/docs/mpi-3.1/mpi31-report.pdf Chap-3

#include <stdio.h>
#include <string.h>
#include "mpi.h"

int main( int argc, char *argv[])
{
    char message[20], recvmesssage[100];
    int myrank;
    MPI_Status status;
    MPI_Init( &argc, &argv );
    MPI_Comm_rank( MPI_COMM_WORLD, &myrank );
    if (myrank == 0) /* code for process 0 */
    {
        strcpy(message, "Hello, there");
        MPI_Send(message, strlen(message)+1, MPI_CHAR, 1, 99, MPI_COMM_WORLD);
    }
    else if (myrank == 1) /* code for process 1 */
    {
        MPI_Recv(recvmesssage, 15, MPI_CHAR, 0, 99, MPI_COMM_WORLD, &status);
        printf("received : %s\n", message);
    }
    MPI_Finalize();
    return 0;
}
```

**Programming
error**

//From <https://www.mpi-forum.org/docs/mpi-3.1/mpi31-report.pdf> Chap-3

```
#include <stdio.h>
#include <string.h>
#include "mpi.h"

int main( int argc, char *argv[])
{
    char message[20], recvmesssage[100];
    int myrank;
    MPI_Status status;
    MPI_Init( &argc, &argv );
    MPI_Comm_rank( MPI_COMM_WORLD, &myrank );
    if (myrank == 0) /* code for process 0 */
    {
        strcpy(message, "Hello, there");
        MPI_Send(message, strlen(message)+1, MPI_CHAR, 1, 99, MPI_COMM_WORLD);
    }
    else if (myrank == 1) /* code for process 1 */
    {
        MPI_Recv(recvmesssage, 10, MPI_CHAR, 0, 99, MPI_COMM_WORLD, &status);
        printf("received : %s\n", recvmesssage);
    }
    MPI_Finalize();
    return 0;
}
```

Runtime error

Message Size

Sender

message (13 bytes)

Receiver

Message (10 bytes)

```
Fatal error in MPI_Recv: Message truncated, error stack:  
MPI_Recv(200).....: MPI_Recv(buf=0x7ffccc37c610,  
count=10, MPI_CHAR, src=0, tag=99, MPI_COMM_WORLD,  
status=0x7ffccc37c5d0) failed  
MPIDI_CH3_PktHandler_EagerShortSend(363): Message from rank 0  
and tag 99 truncated; 13 bytes received but buffer size is 10
```

//From <https://www.mpi-forum.org/docs/mpi-3.1/mpi31-report.pdf> Chap-3

```
#include <stdio.h>
#include <string.h>
#include "mpi.h"

int main( int argc, char *argv[])
{
    char message[20], recvmesssage[100];
    int myrank;
    MPI_Status status;
    MPI_Init( &argc, &argv );
    MPI_Comm_rank( MPI_COMM_WORLD, &myrank );
    if (myrank == 0) /* code for process 0 */
    {
        strcpy(message, "Hello, there");
        MPI_Send(message, strlen(message)+1, MPI_CHAR, 1, 99, MPI_COMM_WORLD);
    }
    else if (myrank == 1) /* code for process 1 */
    {
        MPI_Recv(recvmesssage, 15, MPI_CHAR, 0, 91, MPI_COMM_WORLD, &status);
        printf("received : %s\n", recvmesssage);
    }
    MPI_Finalize();
    return 0;
}
```

No runtime or
compile-time
error

//From <https://www.mpi-forum.org/docs/mpi-3.1/mpi31-report.pdf> Chap-3

```
#include <stdio.h>
#include <string.h>
#include "mpi.h"
```

```
int main( int argc, char *argv[])
{
```

```
    char message[20], recvmesssage[100];
```

```
    int myrank;
```

```
    MPI_Status status;
```

```
    MPI_Init( &argc, &argv );
```

```
    MPI_Comm_rank( MPI_COMM_WORLD, &myrank );
```

```
    if (myrank == 0) /* code for process 0 */
```

```
{
```

```
    strcpy(message, "Hello, there");
```

```
    MPI_Send(message, strlen(message)+1, MPI_CHAR, 1, 99, MPI_COMM_WORLD);
```

```
}
```

```
    else if (myrank == 1) /* code for process 1 */
```

```
{
```

```
    MPI_Recv(recvmesssage, 15, MPI_CHAR, 0, 99, MPI_COMM_WORLD, &status);
```

```
    printf("received : %s\n", recvmesssage);
```

```
}
```

```
    MPI_Finalize();
```

```
    return 0;
```

```
}
```

received : Hello, there

Output

```
MPI_Init( &argc, &argv );
MPI_Comm_rank( MPI_COMM_WORLD, &myrank );
if (myrank == 0)    /* code for process 0 */
{
    strcpy(message, "Welcome");
    MPI_Send(message, strlen(message)+1, MPI_CHAR, 1, 99, MPI_COMM_WORLD);
}
else if (myrank == 1) /* code for process 1 */
{
    MPI_Recv(recvmessage, 15, MPI_CHAR, 0, 99, MPI_COMM_WORLD, &status);
    printf("received : %s\n", recvmessage);
}

printf("%d %lu %lu\n", myrank, strlen(message), strlen(recvmessage));

MPI_Finalize();
```

0 7 0

Received: Welcome

1 0 7

Output for 4 Processes

```
char message[20]; message[0] = '\0';
char rcvmessage[20]; rcvmessage[0] = '\0';
int myrank;
MPI_Status status;
MPI_Init( &argc, &argv );
MPI_Comm_rank( MPI_COMM_WORLD, &myrank );
if (myrank == 0) /* code for process 0 */
{
    strcpy(message, "Welcome");
    MPI_Send(message, strlen(message)+1, MPI_CHAR, 1, 99, MPI_COMM_WORLD);
}
else if (myrank == 1) /* code for process 1 */
{
    MPI_Recv(rcvmessage, 15, MPI_CHAR, 0, 99, MPI_COMM_WORLD, &status);
    printf("received : %s\n", rcvmessage);
}

printf("%d %lu %lu\n", myrank, strlen(message), strlen(rcvmessage));
```

0 7 0

Received: Welcome

1 0 7

2 0 0

3 0 0

```

int main( int argc, char *argv[])
{
    char message[20], recvmesssage[100];
    int myrank;
    MPI_Status status;
    MPI_Init( &argc, &argv );
    MPI_Comm_rank( MPI_COMM_WORLD, &myrank );
    if (myrank == 0)    /* code for process 0 */
    {
        strcpy(message, "Hello, there");
        MPI_Send(message, strlen(message)+1, MPI_CHAR, 1, 99, MPI_COMM_WORLD);
    }
    else if (myrank == 1) /* code for process 1 */
    {
        MPI_Recv(recvmesssage, 15, MPI_CHAR, 0, 91, MPI_COMM_WORLD, &status);
        printf("received : %s\n", recvmesssage);
    }

    printf("%d %lu %lu\n", myrank, strlen(message), strlen(recvmesssage));

    MPI_Finalize();
    return 0;
}

```

```
mpirun -np 2 ./send
```

```
0 12 0
```

```
//1 will be blocked, hence 1
will not print (more later)
```

Multiple Sends and Receives

```
if (myrank == 0) {  
    MPI_Send (buf, count, MPI_INT, 1, 1, MPI_COMM_WORLD),  
    MPI_Send (buf, count, MPI_INT, 1, 2, MPI_COMM_WORLD);  
}  
else if (myrank == 1) {  
    MPI_Recv (buf, count, MPI_INT, 0, 1, MPI_COMM_WORLD, &status),  
    MPI_Recv (buf, count, MPI_INT, 0, 2, MPI_COMM_WORLD, &status);  
}  
printf ("%d %d\n", myrank, count);
```

```
$ mpirun -np 2 ./send 10  
  
0 10  
1 10
```

Multiple Sends and Receives

```
$ mpirun -np 4 ./send 10
```

```
if (myrank == 0) {
```

```
    MPI_Send (buf, count, MPI_INT, 1, 1, MPI_COMM_WORLD),
```

```
    MPI_Send (buf, count, MPI_INT, 1, 2, MPI_COMM_WORLD);
```

```
}
```

```
else if (myrank == 1) {
```

```
    MPI_Recv (buf, count, MPI_INT, 0, 1, MPI_COMM_WORLD, &status),
```

```
    MPI_Recv (buf, count, MPI_INT, 0, 2, MPI_COMM_WORLD, &status);
```

```
}
```

```
printf ("%d %d\n", myrank, count);
```

0	10
1	10
2	10
3	10

Multiple Sends and Receives (Incorrect code)

```
if (myrank == 0) {  
    MPI_Send (buf, count, MPI_INT, 1, 1, MPI_COMM_WORLD),  
    MPI_Send (buf, count, MPI_INT, 1, 2, MPI_COMM_WORLD);  
}  
  
else if (myrank == 1) {  
    MPI_Recv (buf, count, MPI_INT, 0, 1, MPI_COMM_WORLD, &status),  
    MPI_Recv (buf, count, MPI_INT, 0, 1, MPI_COMM_WORLD, &status);  
}  
  
printf ("%d %d\n", myrank, count);
```

```
$ mpirun -np 2 ./send 10  
  
0 10  
// rank 1 will be blocked  
// rank 0 may not be  
blocked (Depending on value  
of count)
```

Multiple Sends and Receives (Unsafe code)

```
if (myrank == 0) {  
    MPI_Send (buf, count, MPI_INT, 1, 1, MPI_COMM_WORLD),  
    MPI_Send (buf, count, MPI_INT, 1, 2, MPI_COMM_WORLD);  
}  
  
else if (myrank == 1) {  
    MPI_Recv (buf, count, MPI_INT, 0, 2, MPI_COMM_WORLD, &status),  
    MPI_Recv (buf, count, MPI_INT, 0, 1, MPI_COMM_WORLD, &status);  
}  
  
printf ("%d %d\n", myrank, count);
```

```
$ mpirun -np 2 ./send 10  
0 10  
1 10
```

Send and Receive (Unsafe code)

```
$ mpirun -np 2 ./send 10
```

```
0 10
```

```
1 10
```

```
MPI_Send (buf, count, MPI_INT, (rank+1)%2, 1, MPI_COMM_WORLD);
```

```
MPI_Recv (buf, count, MPI_INT, (rank+1)%2, 1, MPI_COMM_WORLD,  
&status);
```

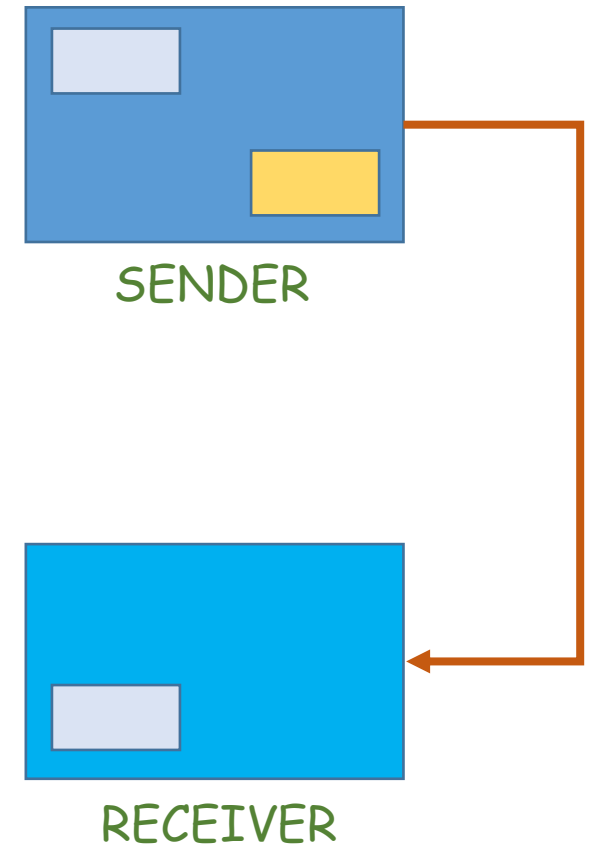
```
printf ("%d %d\n", myrank, count);
```

```
$ mpirun -np 2 ./send 1000000
```

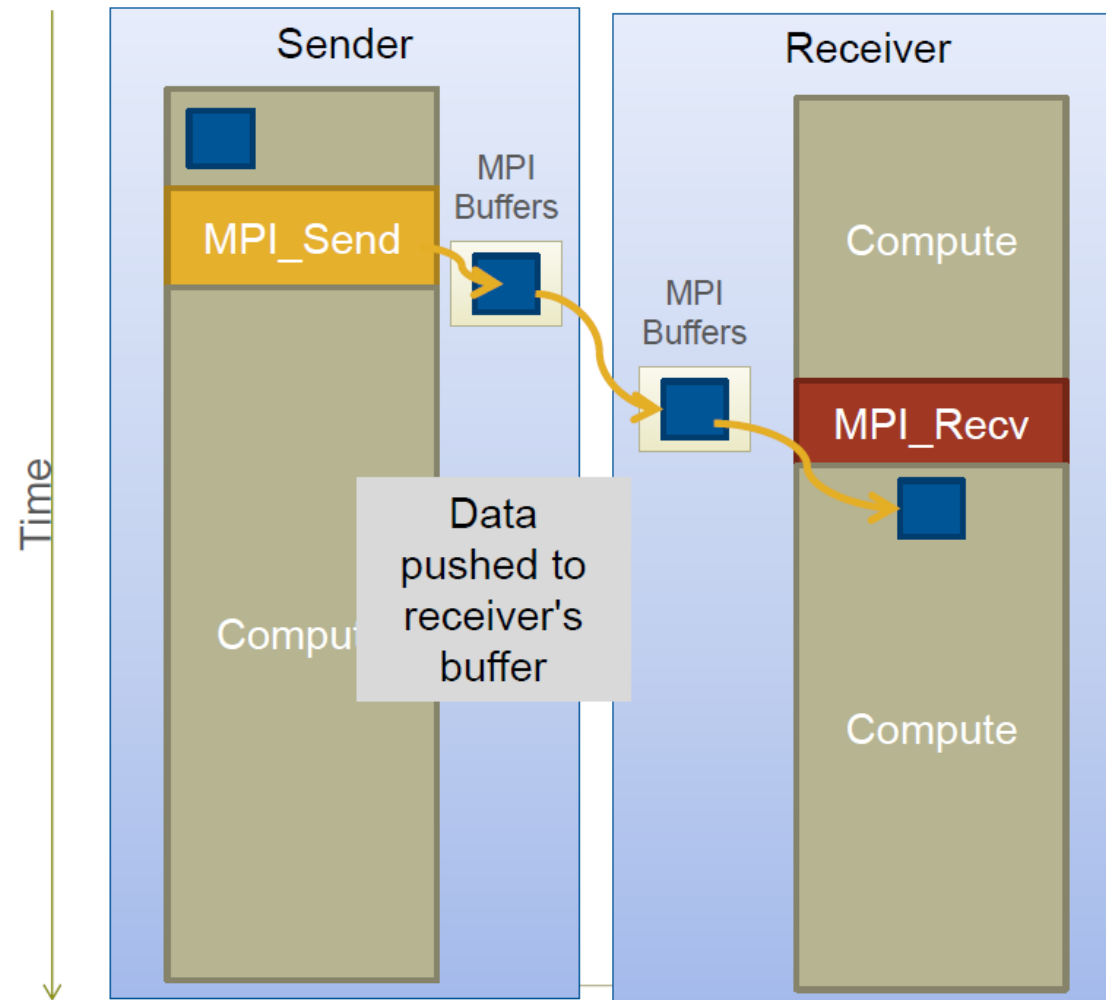
```
<blocked> //because is MPI_Send is blocking (and count is large)
```

MPI_Send (Blocking, Standard Mode)

- Does not return until buffer can be reused
- Message buffering can affect this
- Implementation-dependent



Buffering



[Source: Cray presentation]

MPI_Send is blocking and complete

mpi-forum.org/docs/mpi-5.0/mpi50-report.pdf

Message-Passing Interface Standard

102 / 1189

—

150%

+



13

14

15

3.4 Communication Modes

16

17

18

19

20

The send call described in Section 3.2.1 is *blocking*: it does not return until the *message data* and *envelope* have been safely stored away so that the sender is free to modify the send buffer. The message might be copied directly into the matching receive buffer, or it might be copied into a temporary system buffer.

Multiple Sends and Receives (Unsafe code)

```
if (myrank == 0) {  
    MPI_Send (buf, count, MPI_INT, 1, 1, MPI_COMM_WORLD),  
    MPI_Send (buf, count, MPI_INT, 1, 2, MPI_COMM_WORLD);  
}  
  
else if (myrank == 1) {  
    MPI_Recv (buf, count, MPI_INT, 0, 2, MPI_COMM_WORLD, &status),  
    MPI_Recv (buf, count, MPI_INT, 0, 1, MPI_COMM_WORLD, &status);  
}  
  
printf ("%d %d\n", myrank, count);
```

```
$ mpirun -np 2 ./send 1000000  
<blocked>
```

Eager vs. Rendezvous Protocol

- Eager
 - Send completes without acknowledgement from destination
 - `MPIR_CVAR_CH3_EAGER_MAX_MSG_SIZE` (check output of `mpivars`)
 - Small messages, typically 128 KB (at least in MPICH)
- Rendezvous
 - Requires an acknowledgement from a matching receive
 - Large messages

MPI_Status

```
int MPI_Recv (void *buf, int count, MPI_Datatype datatype, int  
source, int tag, MPI_Comm comm, MPI_Status *status)
```

- Source rank
- Message tag
- Message length
 - MPI_Get_count

```
typedef struct _MPI_Status {  
    int count;  
    int cancelled;  
    int MPI_SOURCE;  
    int MPI_TAG;  
    int MPI_ERROR;  
} MPI_Status, *PMPI_Status;
```

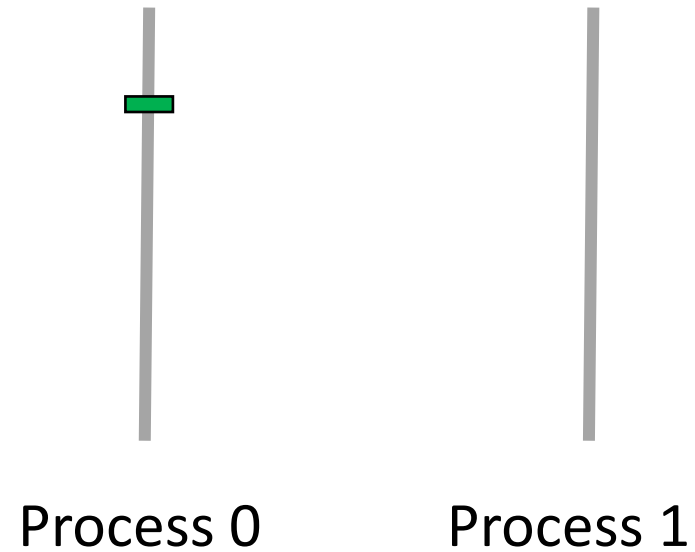
MPI_Get_count (status.c)

status.MPI_SOURCE
status.MPI_TAG

```
if (myrank == 0)    /* code for process zero */
{
    MPI_Send(arr, 100, MPI_INT, 1, 99, MPI_COMM_WORLD);
}
else if (myrank == 1) /* code for process one */
{
    int count, recvarr[numElements];
    MPI_Recv(recvarr, numElements, MPI_INT, 0, 99, MPI_COMM_WORLD, &status);
    MPI_Get_count (&status, MPI_INT, &count);
    printf("Rank %d of %d received %d elements from %d\n", myrank, size, count, status.MPI_SOURCE);
}
```

Rank 1 of 2 received 100 elements from 0

Communication – Message Passing



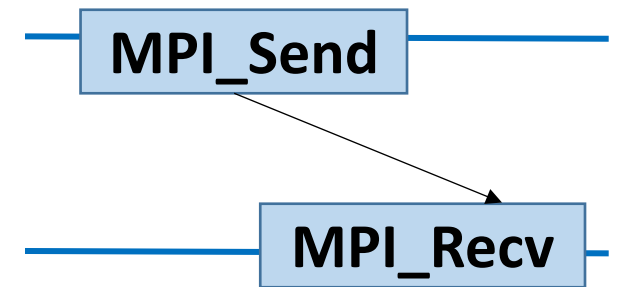
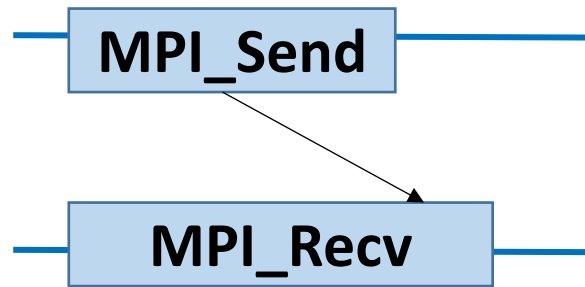
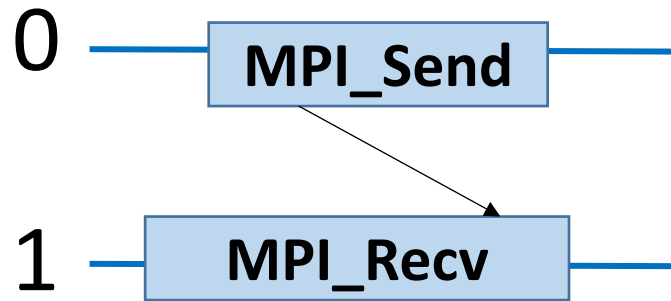
Timing Send/Recv (timingSend.c)

```
MPI_Comm_rank(MPI_COMM_WORLD, &myrank) ;
MPI_Comm_size(MPI_COMM_WORLD, &size);

// initialize data
for (int i=0; i<count; i++)
    buf[i] = myrank+i;

sTime = MPI_Wtime();
if (myrank == 0)
    MPI_Send (buf, count, MPI_INT, 1, 1, MPI_COMM_WORLD);
else
    if (myrank == 1)
        MPI_Recv (buf, count, MPI_INT, 0, 1, MPI_COMM_WORLD, &status);
eTime = MPI_Wtime();
time = eTime - sTime;
```

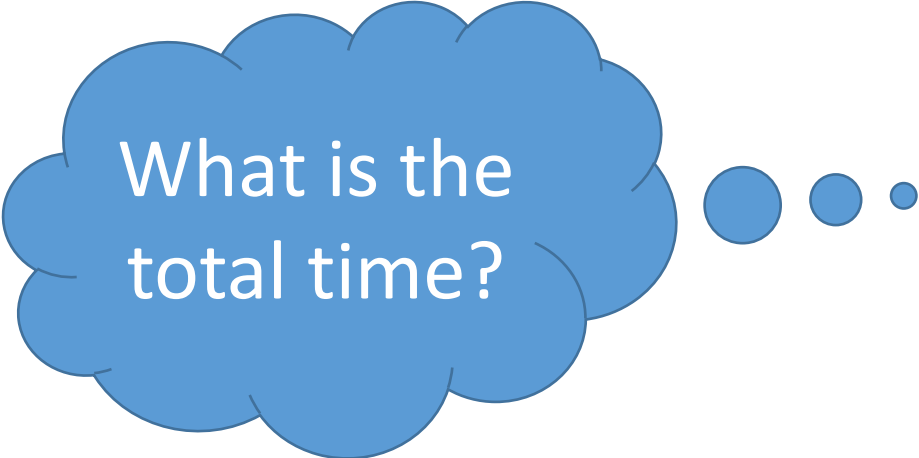
A Few Scenarios



Inspiration: Slides of Prof. Dr. – Ing. Morris Riedel

Timing Output

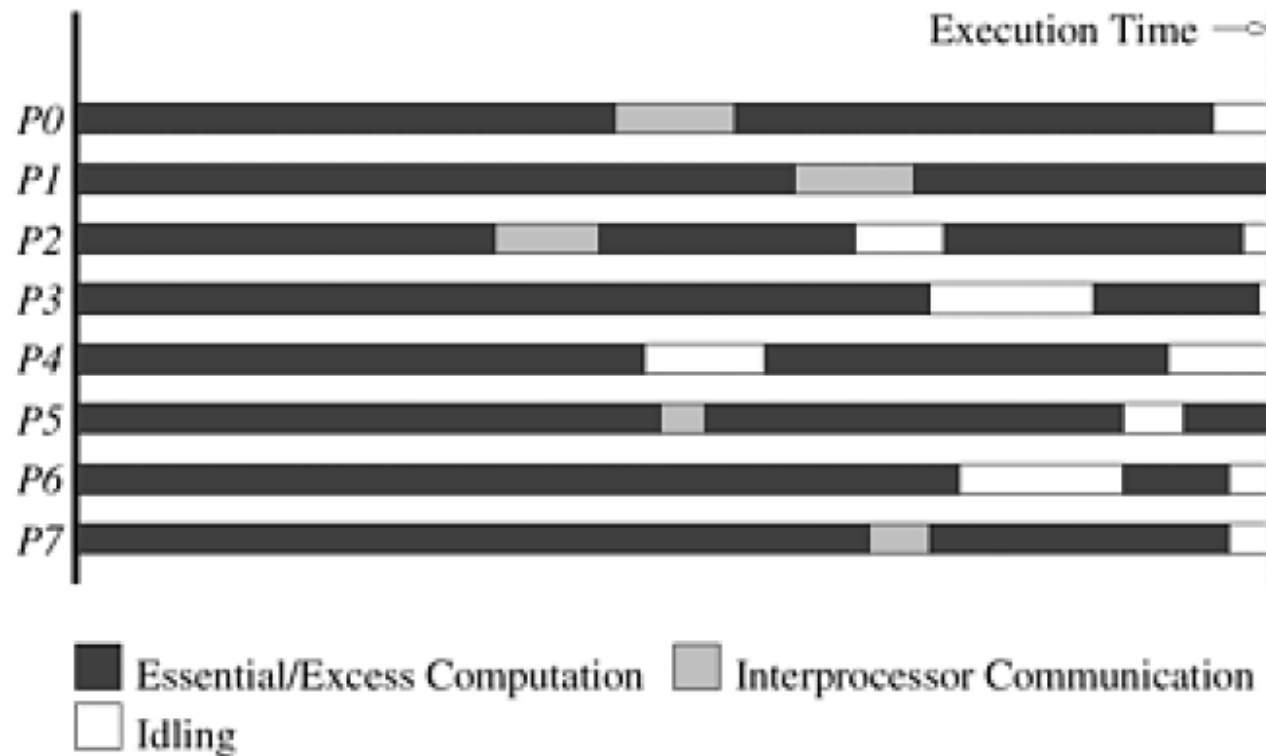
Time: $\max(\text{eTime} - \text{sTime})$
over all processes



What is the
total time?

```
class mpirun -np 2 ./timingSend 100
0.000002
0.000039
class mpirun -np 2 ./timingSend 100
0.000002
0.000009
class mpirun -np 2 ./timingSend 10000
0.000022
0.000039
class mpirun -np 2 ./timingSend 10000
0.000022
0.000030
class mpirun -np 2 ./timingSend 1000000
0.000426
0.000477
class mpirun -np 2 ./timingSend 1000000
0.000398
0.000403
class mpirun -np 2 ./timingSend 100000000
0.052907
0.052171
class mpirun -np 2 ./timingSend 100000000
0.055681
0.056406
```

Performance Variation



Execution Profile/Trace of a Hypothetical Parallel Program

Source: GGKK Chapter 5

```
sTime = MPI_Wtime();
```

```
if (myrank == 0)
```

```
    MPI_Send (buf, count, MPI_INT, 1,  
             1, MPI_COMM_WORLD);
```

```
else if (myrank == 1)
```

```
    MPI_Recv (buf, count, MPI_INT, 0,  
            1, MPI_COMM_WORLD, &status);
```

```
eTime = MPI_Wtime();
```

Variation

1. Node to node
latency/bandwidth and
hence communication time
is different and varies

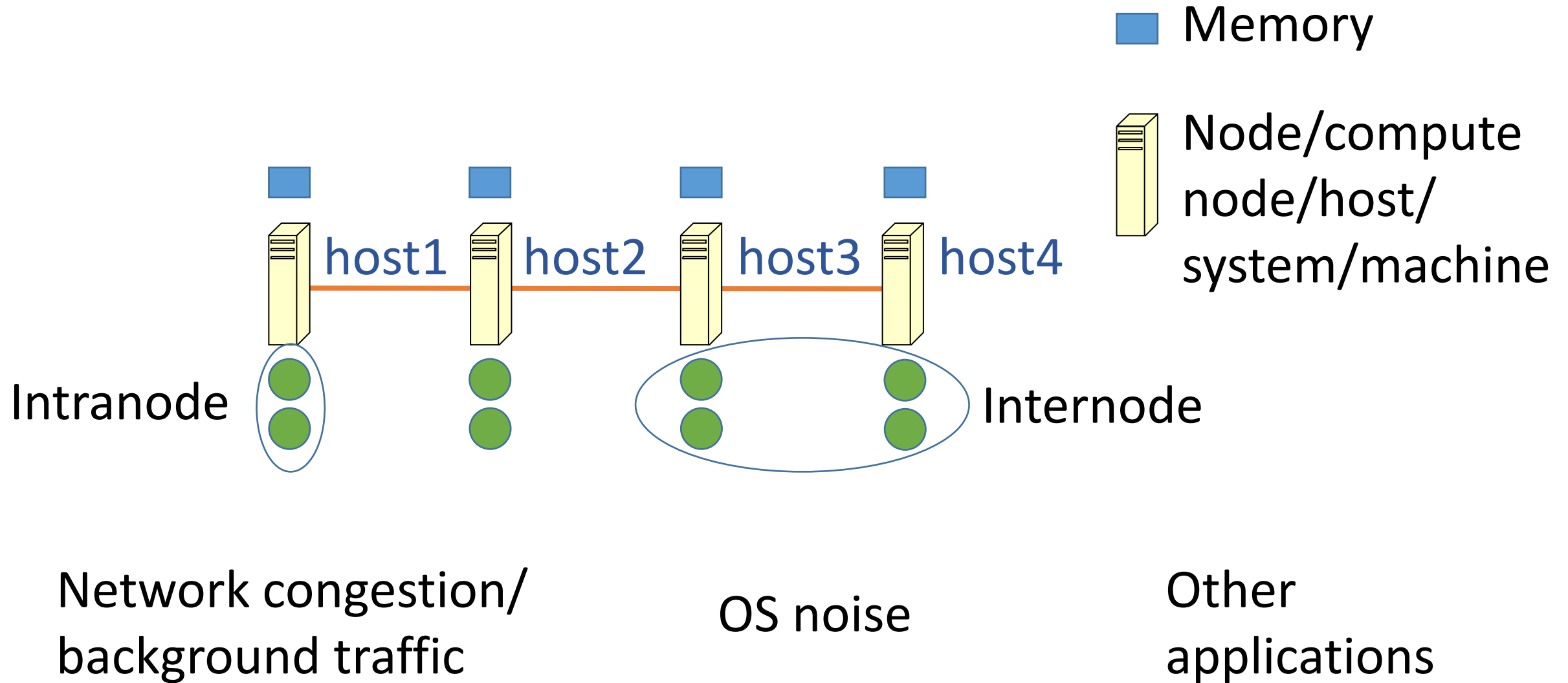
2. Intra-node
communication is typically
faster than inter-node
communication

```
class > mpirun -np 2 ./timingSend 2000
0.000575
0.000634
class > mpirun -np 2 ./timingSend 2000
0.000504
0.000754
class > mpirun -np 2 ./timingSend 2000
0.000568
0.000660
```

```
class >
class >
class > mpirun -np 2 ./timingSend 2000
0.001152
0.000436
class > mpirun -np 2 ./timingSend 2000
0.001096
0.000440
class > mpirun -np 2 ./timingSend 2000
0.001079
0.000420
```

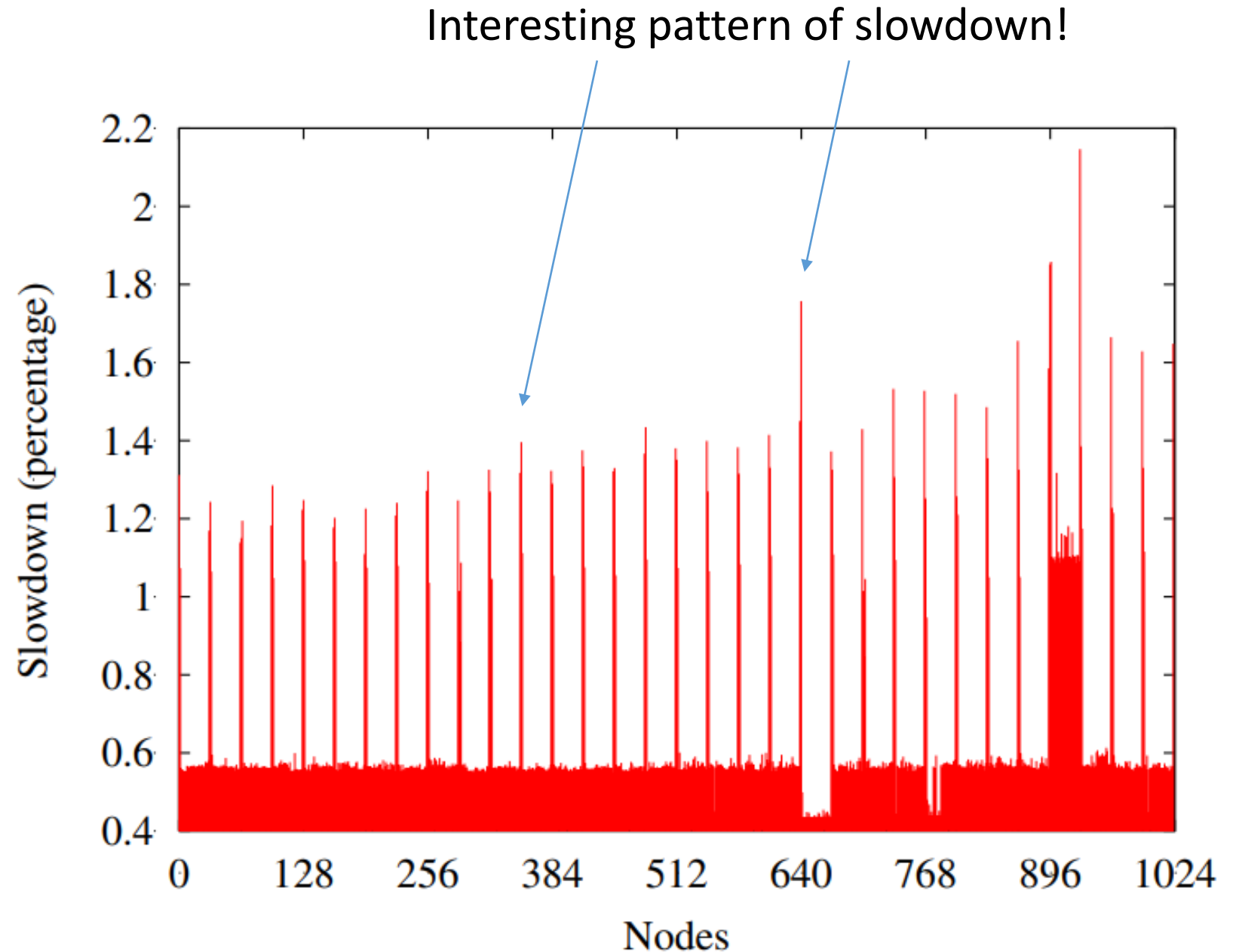
```
class >
class > mpirun -np 2 ./timingSend 2000
0.000008
0.000025
class > mpirun -np 2 ./timingSend 2000
0.000010
0.000010
class > mpirun -np 2 ./timingSend 2000
0.000009
0.000014
```

Sources of Variation

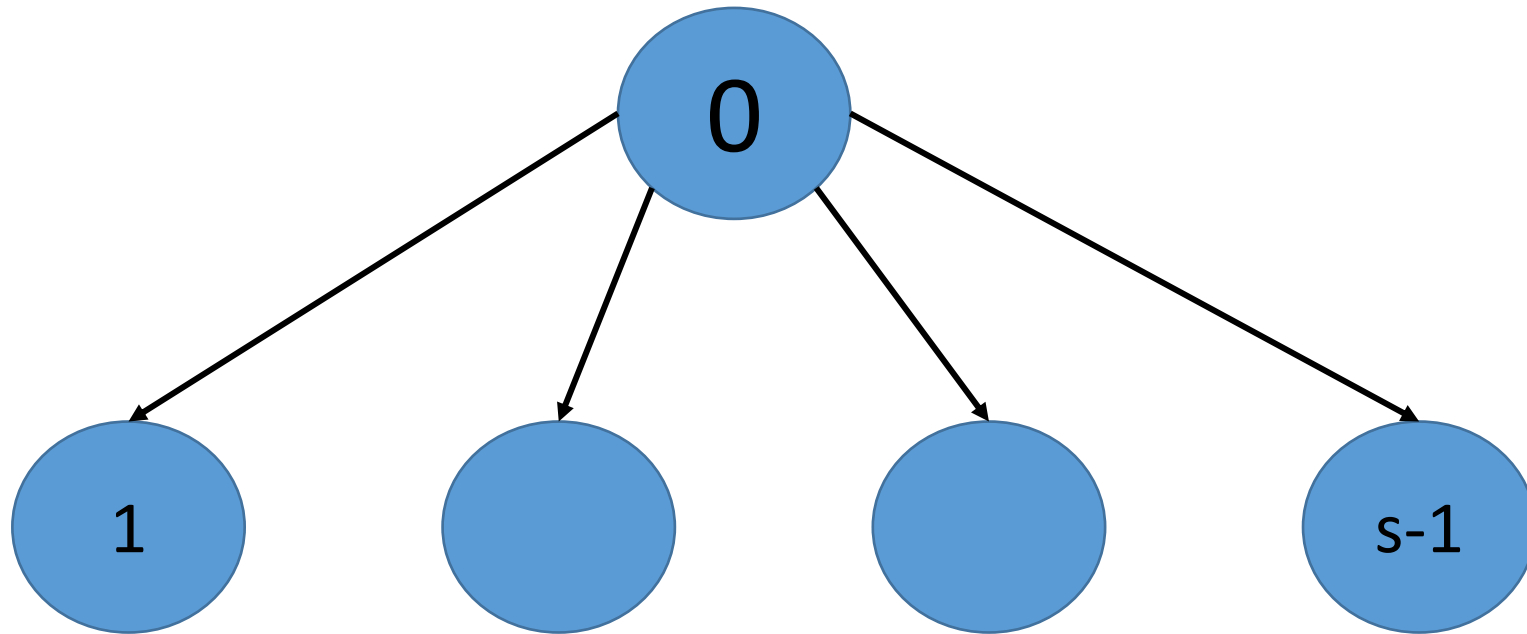


Variability

The Case of the Missing
Supercomputer
Performance: Achieving
Optimal Performance on
the 8,192 Processors of
ASCI Q, IEEE CLUSTER 2010



One-to-many Sends



One-to-many Sends

```
sTime = MPI_Wtime();
if (myrank == 0)
    for (int r=1; r<size; r++)
        MPI_Send (buf, count, MPI_INT, r, r, MPI_COMM_WORLD);
else
    MPI_Recv (buf, count, MPI_INT, 0, myrank, MPI_COMM_WORLD, &status);
eTime = MPI_Wtime();
time = eTime - sTime;

// obtain max time
MPI_Reduce (&time, &maxTime, 1, MPI_DOUBLE, MPI_MAX, 0, MPI_COMM_WORLD);
if (!myrank) printf ("%lf\n", maxTime);
```

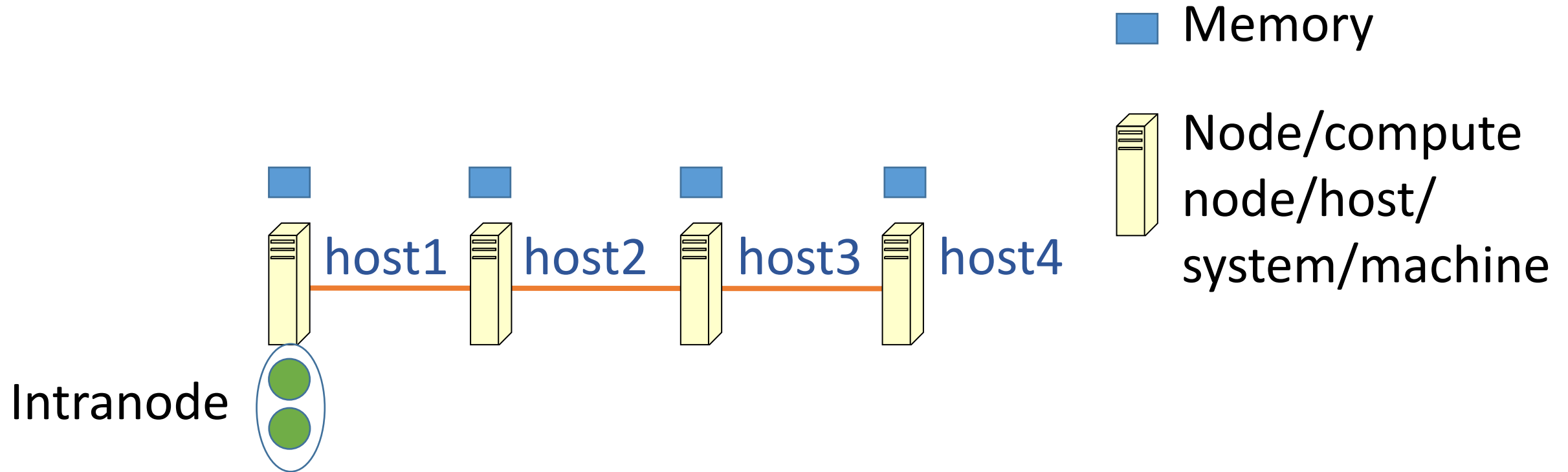
Timing

```
class $ for i in `seq 1 5`; do mpirun -np 4 ./onetomanyseeds 100 ; done
0.000032
0.000014
0.000013
0.000013
0.000012
class $ for i in `seq 1 5`; do mpirun -np 4 ./onetomanyseeds 1000000 ; done
0.002296
0.002655
0.002453
0.002084
0.002210
class $ for i in `seq 1 5`; do mpirun -np 8 ./onetomanyseeds 1000000 ; done
0.004122
0.005187
0.004641
0.004474
0.004451
```



Variability

Execution Time Variability



`mpiexec -n 4./exe`

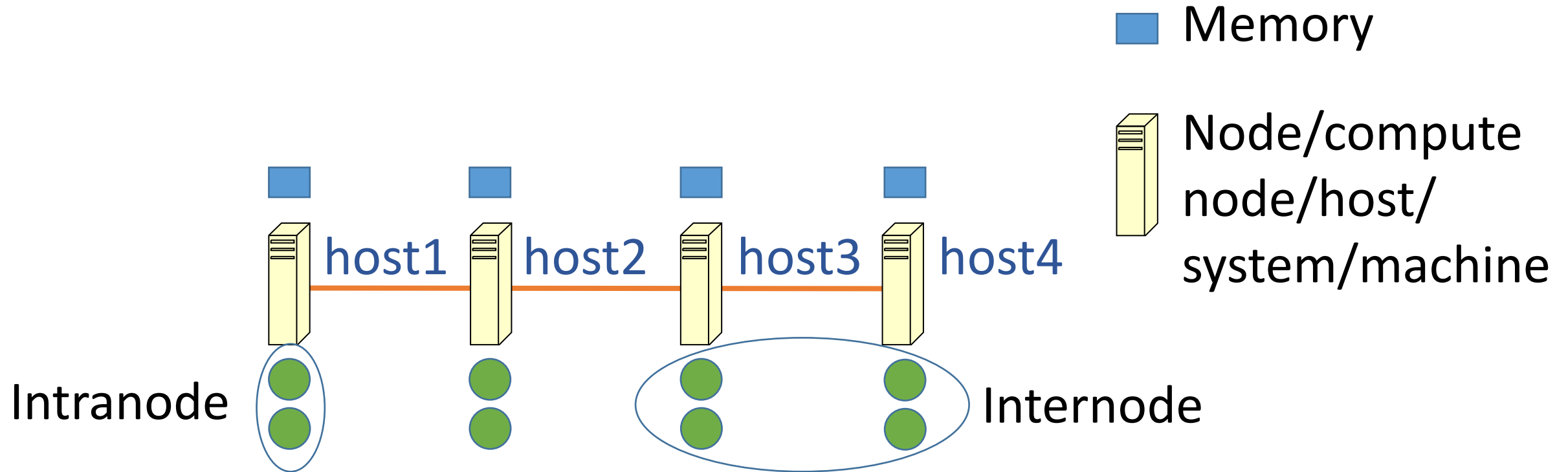
`mpiexec -n 8./exe`

Increase in total time, why?

Output

```
class $ for i in `seq 1 5`; do mpirun -np 8 ./onetomanyseeds 1000000 ; done
0.004122
0.005187
0.004641
0.004474
0.004451
class $ for i in `seq 1 5`; do mpirun -np 8 -hosts csews1:2,csews10:2,csews20:2,csews30:2 ./onetomanyseeds 1000000 ; done
0.205539
0.215736
0.217059
0.214883
0.208585
class $ █
```

Execution Time Variation



`mpixec -n 8 ./exe`

Intraconnect is faster typically

`mpixec -n 8 -hosts host1:2,host2:2,host3:2,host4:2 ./exe`